# Classification

Thursday, November 5, 2026 12:06 PM

# Section 1.2: Classification

AI-IM Class — Lecture Notes [Student Copy]

## 1 From Regression to Classification

In Section 1.1 we predicted a *continuous* output $y \in \mathbb{R}$. Now suppose the output is a **binary label**: $y \in \{0, 1\}$. For example, we might ask:

| **Input** $x$ | **Label** $y$ |
|---|---|
| Tumor size (cm) | 0 = benign, 1 = malignant |
| Hours studied | 0 = fail, 1 = pass |
| Email features | 0 = not spam, 1 = spam |

### 1.1 A Toy Example

Consider six observations, each labelled either class F or class T. We encode the labels numerically as $y = 0$ for F and $y = 1$ for T:

| $x$ | 1 | 2 | 3 | 4 | 5 | 6 |
|---|---|---|---|---|---|---|
| Class | F | F | T | F | T | T |
| $y$ | 0 | 0 | 1 | 0 | 1 | 1 |

Small $x$ is mostly F and large $x$ is mostly T, but the classes *overlap* around $x = 3$ and $x = 4$. This is the region where we should be *uncertain* about the label.

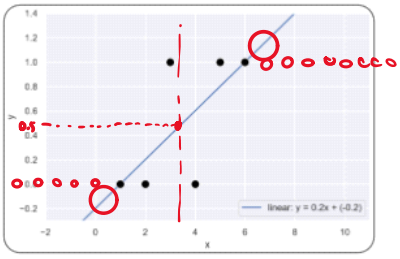



Figure 1: Scatter plot of the toy data with a linear regression line.

**A naive attempt: linear regression.** Since $y$ is now a number, nothing stops us from fitting $\hat{y} = wx + b$ with the closed-form solution from Section 1.1. Here $\bar{x} = 3.5$ and $\bar{y} = 0.5$:

| $i$ | $x_i - \bar{x}$ | $y_i - \bar{y}$ | $(x_i - \bar{x})(y_i - \bar{y})$ | $(x_i - \bar{x})^2$ |
|---|---|---|---|---|
| 1 | −2.5 | −0.5 | 1.25 | 6.25 |
| 2 | −1.5 | −0.5 | 0.75 | 2.25 |
| 3 | −0.5 | 0.5 | −0.25 | 0.25 |
| 4 | 0.5 | −0.5 | −0.25 | 0.25 |
| 5 | 1.5 | 0.5 | 0.75 | 2.25 |
| 6 | 2.5 | 0.5 | 1.25 | 6.25 |
| | | **Sum** | 3.5 | 17.5 |

$$w = \frac{\sum_{i=1}^{n} (x_i - \bar{x})(y_i - \bar{y})}{\sum_{i=1}^{n} (x_i - \bar{x})^2} \qquad b = \bar{y} - w\bar{x}$$

$w =$ $\frac{3.5}{17.5} = 0.2$, $b =$ $0.5 - 0.2 \times 3.5 = -0.2$.

The fitted line is $\hat{y} =$ $0.2x - 0.2$.

**Why not use linear regression?** As a *classifier*, the line is not terrible: thresholding at $\hat{y} = 0.5$ gives the boundary $x = 3.5$, right in the middle of the overlap. The trouble starts when we try to read $\hat{y}$ as a *probability* of class T (see the accompanying marimo notebook):

- **Values outside $[0, 1]$.** At $x = 0$ the line predicts $\hat{y} =$ _____, and at $x = 8$ it predicts $\hat{y} =$ _____. Neither is a valid probability.
- **Constant slope.** Each extra unit of $x$ adds the same 0.2, whether we are deep inside one class or in the uncertain overlap. Intuitively, moving from $x = 3$ to $x = 4$ should change our belief a lot, while moving from $x = 5$ to $x = 6$ (already confidently T) should change it very little.
- **Sensitivity to outliers.** A single far-away point, e.g. $(x, y) = (20, 1)$, flattens the line and drags the 0.5-threshold to the right, so points that were correctly classified before can become misclassified.

## 1.2 What Shape Do We Want?

What we really want is a model that outputs a *probability*:

$$p = P(y = 1 \mid x) \in (0, 1).$$

Probability of $y=1$ given $x$

"$y$ is True"

For the toy data, a sensible curve should:

- stay close to 0 for small $x$ (confidently F),
- stay close to 1 for large $x$ (confidently T),
- rise *smoothly* through the overlap region, passing 0.5 near $x = 3.5$, where we are least certain.

This is an **S-shaped** curve (see the marimo notebook), not a straight line.

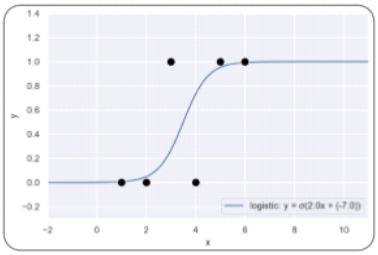



Figure 2: The S-shaped curve that we want to achieve.

## 2 The Sigmoid Function: Motivation via Log-Odds

We would still like to use a linear function $wx + b$: it is simple, interpretable, and we already know how to work with it. The problem is a mismatch of ranges:

$$wx + b \in (-\infty, \infty) \quad \text{but} \quad p \in (0, 1).$$

So instead of forcing $p$ itself to be linear in $x$, we **transform** $p$ into a quantity that lives on all of $\mathbb{R}$, and make *that* quantity linear. We do this in two steps.

**Step 1: probability → odds.** The **odds** of an event with probability $p$ are:

$$\text{odds} = \frac{p}{1-p} \in (0, \infty).$$

This removes the upper limit of 1, but the odds are still positive.

**Step 2: odds → log-odds.** Taking the logarithm maps $(0, \infty)$ onto all of $\mathbb{R}$:

$$\text{log-odds} = \log \frac{p}{1-p} \in (-\infty, \infty).$$

This quantity is also called the **logit** of $p$.

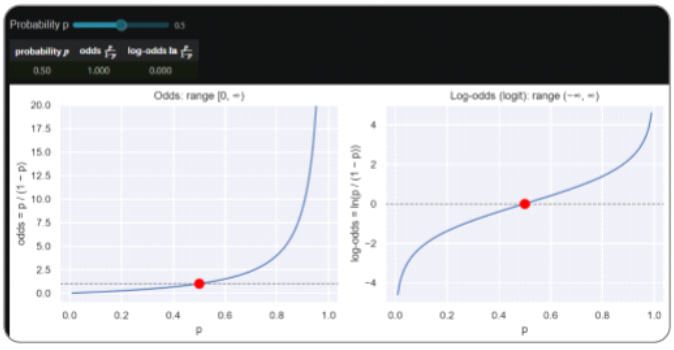



Figure 3: Odd and log-odd transformations of probability.

| $p$ | odds $= \dfrac{p}{1-p}$ | log-odds | Interpretation |
|---|---|---|---|
| 0.1 | 1/9 | $-2.20$ | likely F |
| 0.5 | 1 | 0 | completely unsure |
| 0.9 | 9 | 2.20 | likely T |

The log-odds are symmetric around 0, and $p = 0.5$ maps exactly to 0. Now the ranges match, so we model the log-odds as linear in $x$:

$$\log \frac{p}{1-p} = wx + b. \tag{1}$$

The log-odds is not the only possible choice: any function that maps $(0, 1)$ onto $\mathbb{R}$ would work (for example, the inverse of the normal CDF gives *probit* regression). The log-odds is preferred because it is simple and interpretable: increasing $x$ by one unit multiplies the odds by $e^{w}$.

## 2.1 Solving for $p$

Exponentiating both sides of (1) and solving for $p$:

$$\frac{p}{1-p} = e^{wx+b}$$

$$p = (1-p)\, e^{wx+b}$$

$$p\,(1 + e^{wx+b}) = e^{wx+b}$$

$$\hat{p} = \frac{1}{1 + e^{-(wx+b)}} = \sigma(wx+b)$$

This function $\sigma(z) = \dfrac{1}{1+e^{-z}}$ is called the **sigmoid** (or logistic) function. It is exactly the smooth S-shaped curve we asked for: by making the log-odds linear, the probability automatically becomes S-shaped in $x$.

**Key properties of $\sigma(z)$.**

- Output is always in $(0, 1)$ — valid as a probability.
- $\sigma(0) = 0.5$: the decision boundary is where $wx + b = 0$.
- As $z \to +\infty$, $\sigma(z) \to 1$; as $z \to -\infty$, $\sigma(z) \to 0$.
- Its derivative has the elegant form $\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big)$ (see Appendix A).

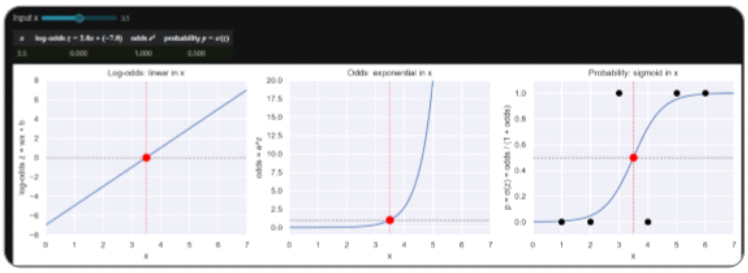



Figure 4: Log-odds, odds, and the sigmoid function.

## 3 The Logistic Regression Model

So far we have used a single feature $x$, and the sigmoid's input is the number $z = wx + b$. With more features, **only** $z$ **changes**: it is still a single number, it just collects more terms.

### 3.1 Two Features: Writing It Out

Suppose each observation has two features $x_1$ and $x_2$ (e.g. hours studied and hours slept). The log-odds are now linear in both:

$$z = b + w_1 x_1 + w_2 x_2 \qquad \hat{p} = \sigma(z) = \frac{1}{1 + e^{-z}}$$

To write this compactly, collect the parameters and the features (with a leading 1 for the bias, as in Section 1.1) into column vectors:

$$\boldsymbol{\theta} = \begin{bmatrix} b \\ w_1 \\ w_2 \end{bmatrix}, \qquad \mathbf{x} = \begin{bmatrix} 1 \\ x_1 \\ x_2 \end{bmatrix}.$$

Then $\boldsymbol{\theta}^\top$ is a *row* vector, and multiplying a row by a column gives back exactly the same sum:

$$\boldsymbol{\theta}^\top \mathbf{x} = \begin{bmatrix} b & w_1 & w_2 \end{bmatrix} \begin{bmatrix} 1 \\ x_1 \\ x_2 \end{bmatrix} = b + w_1 x_1 + w_2 x_2 = z$$

Size check: $(1 \times 3)\,(3 \times 1) = (1 \times 1)$, a **single number**.

**Example 3.1** (Computing $\hat{p}$ with two features)**.** Let $\boldsymbol{\theta} = [-3,\ 1,\ 0.5]^\top$ and consider a student with $x_1 = 2$ hours studied and $x_2 = 4$ hours slept, so $\mathbf{x} = [1,\ 2,\ 4]^\top$.

$$z = \boldsymbol{\theta}^\top \mathbf{x} = \begin{bmatrix} -3 & 1 & 0.5 \end{bmatrix} \begin{bmatrix} 1 \\ 2 \\ 4 \end{bmatrix} = (-3)1 + 1(2) + 0.5(4) = 1$$

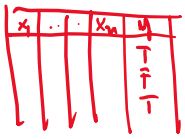
$$\hat{p} = \sigma(z) = \frac{1}{1+e^{-z}} = \frac{1}{1+e^{-1}} \approx 0.73$$

Since $\hat{p} \geq 0.5$, the predicted class is $\hat{y} =$ ____ (pass).

### 3.2 Any Number of Features

For $d$ features the pattern is the same; the vectors just get longer:

$$z = \boldsymbol{\theta}^\top \mathbf{x} = \underbrace{b + w_1 x_1 + w_2 x_2 + \cdots + w_d x_d}_{\text{still one number}}$$




No matter how many features we have, $\boldsymbol{\theta}^\top \mathbf{x}$ is a **single number** (a scalar). The sigmoid only ever sees one number $z$, so everything we learned about $\sigma(z)$ for one feature carries over unchanged. The decision boundary $z = 0$ is a point when $d = 1$, a line when $d = 2$, and a plane (hyperplane) for larger $d$.

Writing the observation index $i$ explicitly, the model is:

$$\hat{p}^{(i)} = \sigma\left(\boldsymbol{\theta}^\top \mathbf{x}^{(i)}\right) = \frac{1}{1 + e^{-\boldsymbol{\theta}^\top \mathbf{x}^{(i)}}}, \tag{2}$$

where $\boldsymbol{\theta} = [b, w_1, \ldots, w_d]^\top$ and $\mathbf{x}^{(i)}$ includes a leading 1 for the bias (same design-matrix convention as Section 1.1).

The predicted class is:

$$\hat{y}^{(i)} = \begin{cases} 1 & \text{if } \hat{p}^{(i)} \geq 0.5 \\ 0 & \text{otherwise.} \end{cases}$$

## 4 Cost Function: Cross-Entropy via MLE

**Why not MSE again?** Technically, we *could* reuse the MSE from Section 1.1, $J(\boldsymbol{\theta}) = \frac{1}{n}\sum_i \left(y^{(i)} - \hat{p}^{(i)}\right)^2$, but it is a poor choice for classification:

1. **Weak learning from big mistakes.** When the model is confidently wrong ($\hat{p}$ near 0 or 1), the sigmoid is flat, so the MSE gradient is tiny and the model barely corrects itself.
2. **Non-convex.** Combined with the sigmoid, MSE is non-convex in $\boldsymbol{\theta}$, so gradient descent can stall on plateaus or local minima.
3. **Wrong noise model.** MSE suits continuous outputs with Gaussian noise; our labels are 0/1, which are naturally modelled as Bernoulli.

Deriving the loss from the Bernoulli likelihood (below) fixes all three.

### 4.1 Setting Up the Likelihood

We model each label $y^{(i)}$ as a Bernoulli random variable with success probability $\hat{p}^{(i)}$:

$$P\left(y^{(i)} \mid \mathbf{x}^{(i)}; \boldsymbol{\theta}\right) = \left(\hat{p}^{(i)}\right)^{y^{(i)}} \left(1 - \hat{p}^{(i)}\right)^{1-y^{(i)}}.$$

Model Prediction

sample No.

P => Probability of observing the outcome of $y^{(i)}$ give $X^{(i)}$ for given parameter $\theta$

$\hat{p}$ => Probability of sucess ($y^{(i)} = 1$)

**Reading the formula.** The model gives $\hat{p}^{(i)}$, the probability that $y^{(i)} = 1$. What we want is the probability of the label we *actually observed*:

$$P\left(y^{(i)} \mid \mathbf{x}^{(i)}; \boldsymbol{\theta}\right) = \begin{cases} \hat{p}^{(i)} & \text{if } y^{(i)} = 1, \\ 1 - \hat{p}^{(i)} & \text{if } y^{(i)} = 0. \end{cases}$$

The Bernoulli formula is just a compact way to write these two cases in one line. The exponents work like a switch:

$$\begin{aligned} y = 1: &\quad \hat{p}^1 (1-\hat{p})^0 &&= \hat{p}^{(i)} \\ y = 0: &\quad \hat{p}^0 (1-\hat{p})^1 &&= 1 - \hat{p}^{(i)} \end{aligned}$$

Anything raised to the power 0 equals 1, so the factor that does not match the observed label simply disappears.

**Example 4.1** (Probability of the observed labels)**.** A model gives the following predictions for four observations:

**Example 4.1** (Probability of the observed labels)**.** A model gives the following predictions for four observations:

| $i$ | Observed $y^{(i)}$ | Predicted $\hat{p}^{(i)}$ | $(\hat{p}^{(i)})^{y^{(i)}}(1-\hat{p}^{(i)})^{1-y^{(i)}}$ |
|---|---|---|---|
| 1 | 0 | 0.1 | $0.1^0 \times 0.9^1 = 0.9$ |
| 2 | 1 | 0.8 | $0.8^1 \times 0.2^0 = 0.8$ |
| 3 | 1 | 0.6 | $0.6^1 (0.4)^0 = 0.6$ |
| 4 | 0 | 0.3 | $0.3^0 (0.7)^1 = 0.7$ |
| 0 | | 0.9 | $(0.9)^0 (0.1)^1 = 0$ |

higher ⇒ model is "good"

Each entry is the probability the model assigns to what *really* happened. A value close to 1 means the model predicted that observation well; a value close to 0 means it was confidently wrong.

Assuming the observations are independent, the probability of seeing *all four* labels is the product:

$\mathcal{L} = 0.9 \times 0.8 \times 0.6 \times 0.7 = 0.302$

Now compare a worse model that predicts $\hat{p} = 0.6, 0.3, 0.4, 0.8$ for the same four observations:

$\mathcal{L} = 0.4 \times 0.3 \times 0.4 \times 0.2 = 0.016$

The better model gives the observed data a much higher probability. **Maximum likelihood estimation (MLE)** simply chooses the parameters $\boldsymbol{\theta}$ that make this product as large as possible.

In general, assuming the $n$ observations are independent, the joint likelihood is:

$$\mathcal{L}(\boldsymbol{\theta}) = \prod_{i=1}^{n} (\hat{p}^{(i)})^{y^{(i)}} (1-\hat{p}^{(i)})^{1-y^{(i)}}.$$

## 4.2 From MLE to the Cross-Entropy Loss

Maximising $\mathcal{L}$ is equivalent to maximising the log-likelihood $\ell = \log \mathcal{L}$ (log is monotone):

Regression

$y^{(i)}$ $\hat{y}^{(i)}$ → residual/MSE → high → bad Model; Low → good Model

Classification

$y^{(i)}$ $\hat{p}^{(i)}$ → P/L → high → good Model; Low → bad Model

| $y^{(i)}$ | $\hat{p}^{(i)}$ | $(\hat{p}^{(i)})^{\hat{y}^{(i)}}(1-\hat{p}^{(i)})^{1-\hat{y}^{(i)}}$ |
|---|---|---|
| 0 | 0.6 | $1-0.6 = 0.4$ |
| 1 | 0.3 | 0.3 |
| 1 | 0.4 | 0.4 |
| 0 | 0.8 | $1-0.8 = 0.2$ |

$$\ell(\boldsymbol{\theta}) = \sum_{i=1}^{n} \left[ y^{(i)} \log \hat{p}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)}) \right]$$

Minimising the **negative** log-likelihood (and averaging over $n$) gives the **Binary Cross-Entropy (BCE)** loss:

$$J(\boldsymbol{\theta}) = -\frac{1}{n} \sum_{i=1}^{n} \left[ y^{(i)} \log \hat{p}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{p}^{(i)}) \right]$$

**Why cross-entropy?** When $y^{(i)} = 1$, only the first term $-\log \hat{p}^{(i)}$ matters: we are penalised heavily if $\hat{p}^{(i)}$ is close to 0 (confident and wrong). When $y^{(i)} = 0$, only the second term $-\log(1 - \hat{p}^{(i)})$ matters. The two terms switch on and off depending on the true label.

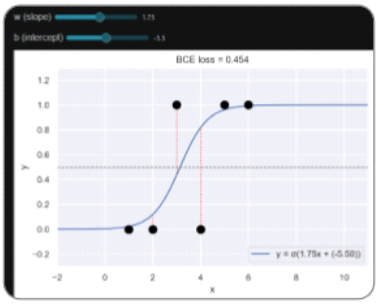



Figure 5: Interactive plot showing BCE loss.

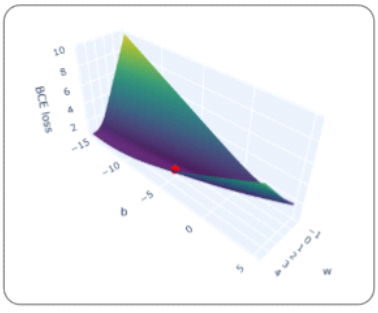



Figure 6: BCE loss surface

## 5 The Gradient and Why We Need Gradient Descent

Unlike MSE in linear regression, there is **no closed-form solution** for the minimiser of $J(\boldsymbol{\theta})$. The cross-entropy loss is convex (it has a unique global minimum), but we cannot invert the normal equations here because the sigmoid is non-linear.

Taking the gradient with respect to $\boldsymbol{\theta}$ (using the chain rule and the identity $\sigma'(z) = \sigma(z)(1-\sigma(z))$):

$$\nabla_{\boldsymbol{\theta}} J = \frac{1}{N} \sum_{i=1}^{n} \left( \hat{p}^{(i)} - y^{(i)} \right) x^{(i)}$$

**Remarkably clean form.** The gradient has exactly the same structure as in linear regression — it is a weighted sum of the input vectors, where the weights are the *prediction errors* $(\hat{p}^{(i)} - y^{(i)})$. The sigmoid "absorbed" all the complexity through the chain rule.

Because there is no closed form, we use **gradient descent**: starting from an initial $\boldsymbol{\theta}^{(0)}$, we repeatedly update

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \alpha \nabla_{\boldsymbol{\theta}} J,$$

where $\alpha > 0$ is the **learning rate**. Gradient descent will be covered in detail in the next section.

## 6 Visualising the Decision Boundary

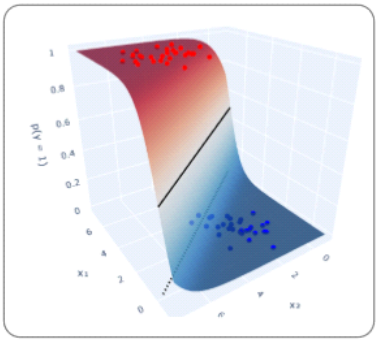


Figure 7: Visualising the decision boundary in 2D

The figure shows $\hat{p} = \sigma(b + w_1 x_1 + w_2 x_2)$ as a surface over the $(x_1, x_2)$ plane: the red points (class 1) sit on the high plateau ($\hat{p} \approx 1$), the blue points (class 0) on the low plateau ($\hat{p} \approx 0$), and the surface rises smoothly between them. The solid black line marks where the surface crosses $\hat{p} = 0.5$; its shadow on the floor (dotted) is the **decision boundary**.

**Logistic regression draws a straight line between the classes.** The boundary is where the model is completely unsure:

$$\hat{p} = 0.5 \iff z = 0 \iff \underline{\qquad\qquad}.$$

This is a *linear* equation in $x_1$ and $x_2$, so the boundary is a **straight line**. Every point on one side has $z > 0$ (predicted class 1), and every point on the other side has $z < 0$ (predicted class 0).

**Example 6.1** (Boundary for the two-feature model)**.** With $\boldsymbol{\theta} = [-3,\ 1,\ 0.5]^\top$ from Section 3, the boundary is

$$-3 + x_1 + 0.5\,x_2 = 0 \implies x_2 = \underline{\qquad\qquad}.$$

The student with $(x_1, x_2) = (2, 4)$ has $z = 1 > 0$, so they lie on the class-1 side of the line, which agrees with $\hat{p} \approx 0.73$.

**What the sigmoid adds.** The line itself comes only from $z = \boldsymbol{\theta}^\top \mathbf{x}$. The sigmoid turns the *distance* from the line into a confidence: points near the line have $\hat{p} \approx 0.5$, and points far away have $\hat{p}$ close to 0 or 1. Scaling all of $\boldsymbol{\theta}$ by the same factor keeps the line in place but makes the transition steeper.
**Limitation.** With $d$ features the boundary is a hyperplane, but it is always *flat*. If the classes cannot be separated by a straight line (or plane), logistic regression cannot classify them correctly. We return to this in Section 1.3.

## 7 Summary

| | |
|---|---|
| **Model** | $\hat{p} = \sigma(\boldsymbol{\theta}^\top \mathbf{x})$ |
| **Loss function** | BCE: $J = -\frac{1}{n}\sum\big[y \log \hat{p} + (1-y)\log(1-\hat{p})\big]$ |
| **Gradient** | $\nabla_{\boldsymbol{\theta}} J = \frac{1}{n}\sum(\hat{p}^{(i)} - y^{(i)})\,\mathbf{x}^{(i)}$ |
| **Solution method** | Gradient descent (no closed form) |
| **Works well when** | The decision boundary is (approximately) linear |

## A Derivative of the Sigmoid Function

Let $\sigma(z) = (1 + e^{-z})^{-1}$. By the chain rule:

$$\sigma'(z) = -\frac{-e^{-z}}{(1+e^{-z})^2} = \frac{e^{-z}}{(1+e^{-z})^2}.$$

Factor out $\dfrac{1}{1+e^{-z}}$ from the numerator and denominator:

$$\sigma'(z) = \frac{1}{1+e^{-z}} \cdot \frac{e^{-z}}{1+e^{-z}} = \sigma(z) \cdot \frac{1+e^{-z}-1}{1+e^{-z}} = \sigma(z)\big(1-\sigma(z)\big).$$

$$\sigma'(z) = \sigma(z)\big(1 - \sigma(z)\big)$$

This compact form makes the chain-rule calculations in gradient descent particularly clean.